课程笔记：语言模型智能体的工具使用

Bowen-mantou（整理）

2026-10-23

视频作者/频道：B 站「常青藤名校课」· CMU 11-768 AI Agents（英文原声字幕版）
发布日期：2026-09-09
视频时长：57:22
视频链接：https://www.bilibili.com/video/BV1a6Yx62EH4?p=16
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 引言：工具是智能体与语言模型的分水岭 **2**
1.1 本章小结 . . . 2

2 工具的两大收益：扩展能力与便利化 **2**
2.1 工具的分类：从文本回复到自定义 API . . . 2
2.2 本章小结 . . . 3

3 三种工具调用范式：函数、代码与并行 **3**
3.1 函数调用：给模型一张 API 清单 . . . 3
3.2 代码即元工具：CodeAct 的思路 . . . 3
3.3 代码调用的代价：为什么不能一切皆代码 . . . 3
3.4 并行工具调用：一次生成，多个动作 . . . 4
3.5 本章小结 . . . 4

4 工具调用在模型内部：token、协议与模板 **4**
4.1 工具调用是 token 序列 . . . 4
4.2 训练视角：格式 token 从哪来 . . . 4
4.3 每个模型的协议都不一样 . . . 4
4.4 本章小结 . . . 5

5 从模型输出到真正执行：解析、分派与结果配对 **5**
5.1 本章小结 . . . 5

6 受限解码：让不合法的工具调用不可能生成 **5**
6.1 问题：模型不保证输出合法 JSON . . . 5
6.2 形式语言视角：从正则文法等价到下推自动机 . . . 6
6.3 xGrammar：在 logits 上做掩码 . . . 6
6.4 唯一的漏洞：token 预算耗尽 . . . 6
6.5 本章小结 . . . 6

7 工具接口的工程化：REST API 与 MCP **7**
7.1 三种把工具接进 agent 的方式 . . . 7
7.2 MCP 真正的价值：多一层凭证隔离 . . . 7
7.3 本章小结 . . . 7

8 工具使用的评测：把「会调工具」变成可测量的指标 **7**
8.1 本章小结 . . . 8

9 总结与延伸 **8**

1 引言：工具是智能体与语言模型的分水岭

语言模型本质上是一台「预测下一个 token」的机器：它的参数在训练结束后就冻结了，既看不到今天的信息，也无法在真实世界里动手。工具（tool）的意义，就是给这台机器开一个通往外部程序的接口——模型输出一段结构化的调用请求，外部程序真正执行，再把结果作为观察（observation）回填到上下文里。主讲人把这一点讲得很直接：**工具使用是智能体（agent）与语言模型最根本的区别**，没有工具调用，就只剩下「一次性的文本问答」。

一个容易被忽略的细节是：**工具调用的形式是标准化的，但工具下游能做什么完全由你决定**。搜索、计算器、浏览器、Python 解释器、任意第三方 API，甚至调用一个电商接口替用户下单，都共用同一套「模型发出调用、系统执行、结果回填」的协议。标准化发生在协议层，能力和风险则发生在实现层。

本讲路线图

为什么需要工具（两类收益）→ 工具的分类与形态 → 函数调用、代码调用、并行调用三种范式 → 工具调用在模型内部如何用 token 表达与训练 → 解析、校验、分派、结果配对 → 受限解码（让非法调用不可能生成）→ REST 与 MCP 两种接口工程 → 工具使用如何评测。

1.1 本章小结

工具是智能体接入外部世界的唯一接口；本讲从「为什么需要」讲到「工程上怎么接、怎么保证不出错、怎么评测」。理解这一点，是理解后续长上下文、记忆、规划等所有 agent 能力的前提。

2 工具的两大收益：扩展能力与便利化

主讲人引用了课程阅读材料（Neubig、Fried 与 Zora Wang 合著的工具使用综述）中的分类，把工具的价值分成两类，这个二分法值得记住。

第一类是「扩展」（extending）：语言模型在原理上做不到的事情，靠工具能做到。最典型的是时效性——模型的参数停留在训练截止时刻，问它「四个月前的今天发生了什么」它只能编；而调用一个外部时间/检索函数就能拿到真实答案。同一类还包括访问私有状态（用户资料、日历、订单）以及对外部环境做出改变（浏览网页、调用某个 API 下单）。

第二类是「便利化」（facilitating）：语言模型其实能做，但让工具做更快、更可靠、更便宜。例如多位数乘法，模型要靠长链推理一步步算，速度慢且可能算错，而计算器可以瞬间给出精确结果。一句话概括：**扩展是「不能变成能」，便利化是「能变成又快又准」**。主讲人在课上给出的判据是：只有当工具的收益超过它的延迟、成本、失败风险时，才应该调用工具。

2.1 工具的分类：从文本回复到自定义 API

课堂上把常见工具归成了几大类，几乎任何新工具都能放进其中某一类：

1. **文本回复（textual responses）**：这是最「无聊」的一类，模型直接给出答案。实现上有两种方式——提供 `finish(text)` 这样的结束工具，或者干脆让模型不调用任何工具、结束 agent 循环。

2. **信息检索（web search / RAG）**：检索外部信息再基于其生成答案。主讲人在这里纠正了一个流行论调：所谓「RAG 已死」并不成立，**RAG 恰恰是因为无处不在才让人觉得它「死了」**

——检索增强已经是几乎所有应用的默认形态。

3. **代码执行（code execution）**：把一段代码交给解释器去算，适合精确计算和数据处理。
4. **图像生成（image generation）**：与文本类工具不同，它的接口是「图像」。一个研究味很浓的细节是：模型往往会在调用前把用户 query 大幅改写/扩展，甚至下载图片时你会看到与用户原话完全不同的 caption——**改写 query 本身就是工具调用能力的一部分**。
5. **自定义函数（custom functions）**：如「把香蕉、牛奶、咖啡加进购物车」这类业务 API，本质上是把某个垂直能力包装成一个函数。

课堂用 ChatGPT 的演化说明这件事有多普遍：它最初只是「文本进、文本出」，如今已经是一个「everything app」——写代码、渲染 3D、生成图像、生成 PDF、连接 Slack 等第三方服务。用户感知到的是功能列表，工程上则是背后挂载的一长串工具与 API。

2.2 本章小结

工具分两类价值：扩展模型做不到的能力（时效、私有状态、真实动作），以及替代模型做得不够好的能力（精确计算）。工具形态可以归为文本回复、检索、代码执行、图像生成、自定义 API 五类；「RAG 无处不在」和「query 改写」是两个值得记住的细节。

3 三种工具调用范式：函数、代码与并行

3.1 函数调用：给模型一张 API 清单

最直接的范式是：把 50–70 个 API 函数连同各自的 schema 一起交给模型，模型每一步选一个（或并行选多个）函数调用。这是当前工业界最主流的做法，优点是权限边界清晰、每个调用都能审阅；缺点是当任务需要「查天气 → 查价格 → 换算汇率 → 再查价格」这种多步组合时，交互轮数会变得很多，延迟和成本随之上升。

3.2 代码即元工具：CodeAct 的思路

第二种范式把代码本身当作工具。代码是极其表达力强的媒介：一段程序可以看作一棵函数调用树，还带循环、条件与变量；调用 `pandas` 就等于一次性打开了整个库的工具箱。因此让 agent 直接「写一段程序调用工具」，往往比逐个 API 调用更自然。

主讲人举了一个经典例子：判断在美国、日本、德国、印度买某款手机哪个最便宜。按传统方式，模型要一次一次调用「查汇率」「查手机价格」「算税」「换算」等工具，来回很多轮；而写成一段程序，一次生成就能把整套流程表达完——这是 CodeAct 的核心动机。论文报告的结果也很直接：**任务成功率上升、平均交互轮数下降**，而且这个收益不限于「本来就需要写代码」的任务。

3.3 代码调用的代价：为什么不能一切皆代码

代码能力越强，风险面越大。课堂讨论了四类问题：

1. **死循环**：模型可能写出无限循环，系统必须有超时/中断机制兜底。
2. **供应链安全**：模型可能引入被投毒的第三方库，一旦中招，整个 agent 系统会被攻破——这是当前 agent 安全里非常现实的一类风险。

3. **难约束**：动作空间大、后果不可预测，很难像 API 那样做细粒度权限校验。

4. **资源影响**：代码可以吃掉大量内存、磁盘和 CPU，需要显式限制资源。

因此，一旦把工具调用升级为「程序化调用」，就必须配套沙箱（sandbox）与资源限额。课程把它留到安全专题展开，但结论先给出来了：**能力越强，容器就要越硬**。

3.4 并行工具调用：一次生成，多个动作

当多个工具调用彼此独立时（例如同时查天气、查日历、查航班状态），可以让模型在单次响应里发出多个工具调用块，系统用 `asyncio.gather` 一类原语并发执行，再把多个结果一起回填。反之，有依赖关系的调用（先查用户 ID，再用 ID 查订单，再退款）必须串行——**能否并行取决于调用之间是否存在数据依赖**。

一个反直觉的现象是：**按任务完成成本算，更贵的模型反而可能更便宜**。原因有二：更强的模型一次选对方案、少走弯路；更强的模型并行工具调用能力更好，能一次读多个文件、同时看多张图。主讲人指出，并行调用的能力在最近半年才明显变好，一个重要推动力是强化学习——训练时对完成时长/冗长程度施加惩罚，会天然激励模型把独立调用并行化。

3.5 本章小结

函数调用胜在可控，代码调用胜在表达力与轮数，并行调用胜在延迟。三者的选择标准是「权限、依赖、时延」三件事：权限敏感用函数，多步组合用代码，相互独立则并行；而代码调用必须配沙箱。

4 工具调用在模型内部：token、协议与模板

4.1 工具调用是 token 序列

工具调用不是「另一套 API」，而是模型生成的 token 序列。普通文本续写是「今天天气是晴天」；工具续写是 `<tool_call>get_weather(...)</tool_call>` 这样的结构；实际输出里通常两者兼有——先对用户说一句话，再发出工具调用。这正是 ReAct 循环在今天的落地方式：**ReAct 不再是写在外部代码里的 while 循环，而是一次 completion 里同时包含思考、发言与调用**。对带长思考的推理模型，输出还可能分成「隐式的 thinking token」「给用户看的 message」「真正的 tool call」三段。

4.2 训练视角：格式 token 从哪来

课堂提问：「这些特殊 token 需要在预训练时就进词表吗?」主讲人的回答是：不一定。更常见的做法是在**中训练（mid-training）**阶段引入工具调用格式，然后才做强化学习；预训练在整张互联网上做时，词表里未必有这些 token。这一点对做后训练的人很重要：**工具调用格式是一项需要专门训练的能力，而不是凭空泛化出来的**。

4.3 每个模型的协议都不一样

同样的工具调用，不同模型的编码格式不同：Qwen 用 ChatML 风格的特殊标记；Mistral 有自己的一套 tool call 标记；DeepSeek 把它叫 function call，用自研的 DSML（DeepSeek Markup Language）标记；Anthropic 又是另一种格式。工程结论有两条：

1. **微调时必须匹配基座模型的原生格式**——不要试图让模型学一个它没见过的格式；
2. **解析代码不能跨模型复用**——为 DeepSeek 写的解析器未必能解析 Qwen 的输出，反之亦然。

好消息是这类模板已经基本标准化（Hugging Face 的 `apply_chat_template` 会按模型注入正确格式），但知道底层发生了什么，能避免很多「看起来莫名奇妙」的 bug。

4.4 本章小结

工具调用是在 token 层面生成的结构化文本，格式 token 通常在中训练阶段引入；不同模型协议不同，微调必须匹配原生模板，解析器不能跨模型照搬。

5 从模型输出到真正执行：解析、分派与结果配对

模型吐出调用之后，agent 框架要完成一条固定的流水线：

1. **解析（parse）**：把模型输出解析成「工具名 + 参数」；
2. **注册与解析（register / resolve）**：维护一张「工具名 → 可执行函数」的映射表，框架通常提供注册新工具的接口；
3. **校验（validate）**：检查工具是否存在、参数是否符合 schema；
4. **执行（execute）**：调用真实函数，捕获异常；
5. **返回观察（observation）**：把结果包装成一条带调用 ID 的消息，回填进对话历史。

这里有两类失败要区分：**校验失败**是模型选错了工具或参数不合法；**执行失败**是参数看起来合法、但运行时出错——典型例子是「执行 Python」工具，任何字符串都能通过校验，但真跑起来可能报语法错误或异常。工程上要把两者分开处理，前者可以提示模型改正，后者更适合把异常信息作为观察回填。

另一个容易踩的坑是**调用 ID 与结果 ID 的配对**。每次工具调用会被分配一个 ID，对应的结果必须带同一个 ID；并行调用时靠它区分「哪个结果对应哪个调用」。多数模型对此宽容，但 Anthropic 的接口在遇到「有调用、无结果」的历史时会直接拒绝继续生成。这对应一类现实故障：agent 发出调用后程序崩溃、结果没写回，恢复会话时历史不完整——修复方式是在恢复时补齐缺失的结果（哪怕是错误信息），而不是把残缺历史直接发给模型。

5.1 本章小结

工具调用是一条「解析 → 校验 → 分派 → 执行 → 回填」的流水线；要区分校验失败与执行失败，并保证每个调用都有 ID 匹配的结果，否则会制造出模型无法继续的残缺历史。

6 受限解码：让不合法的工具调用不可能生成

6.1 问题：模型不保证输出合法 JSON

语言模型不保证生成格式正确的工具调用——缺一个右括号的 JSON 就是典型例子。很多人第一反应是「写个脚本把括号补上」，但这既不可靠也不优雅。正确思路是把约束前置到解码阶段。

工具调用的合法性可以拆成三层约束：

1. **语法（syntax）**：必须是合法 JSON；
2. **形状（shape）**：必填字段齐全；
3. **类型与单位（types / units）**：例如 `city` 必须是字符串，`units` 只能是摄氏度或华氏度。这三层都可以用 JSON schema 表达。

6.2 形式语言视角：从正则文法等价到下推自动机

课堂用形式语言理论解释了为什么这件事可以做到「生成即合法」。正则表达式对应**正则文法**，可以被**有限状态自动机（FSA）**识别：每读入一个 token，状态转移一次。但 JSON schema 不是正则语言——JSON 有嵌套括号、逗号分隔的成员，存在递归结构，需要用**上下文无关文法（CFG）**描述；识别 CFG 需要比有限状态自动机更强的机器：**下推自动机（PDA）**，也就是在状态机上再加一个栈，用入栈/出栈记住当前嵌套深度。

6.3 xGrammar：在 logits 上做掩码

工程上最流行的实现是 **xGrammar**（CMU 机器学习系的工作，已被广泛集成进各家推理库）。其核心机制是：每生成一个 token 之前，查当前的语法状态，允许的 token 保留原 logits，不允许的 token 置为负无穷，再重新归一化：

$$p'(x_t = v \mid x_{<t}) = \frac{\exp(z_v) \cdot \mathbb{1}[v \in \mathcal{V}_{\text{valid}}(s_t)]}{\sum_{v'} \exp(z_{v'}) \cdot \mathbb{1}[v' \in \mathcal{V}_{\text{valid}}(s_t)]},$$

其中 z_v 是模型对 token v 的原始 logits，s_t 是生成到第 t 步时的语法状态（包含下推自动机的栈），$\mathcal{V}_{\text{valid}}(s_t)$ 是在该状态下合法的 token 集合，$\mathbb{1}[\cdot]$ 是指示函数。这样生成的序列在语法上永远合法。

性能上，xGrammar 做了两类优化：先预计算「永远合法」和「永远非法」的词表项，只有随上下文变化的 token 才在运行时按栈状态计算；同时把 CPU 上的语法计算与 GPU 上的推理重叠起来，从而把约束解码的额外开销压得很低。

6.4 唯一的漏洞：token 预算耗尽

即使开启受限解码，仍有一种情况会生成不完整的 JSON：模型的输出 token 上限到了，而当前状态虽然仍在一条合法路径上，却还没走到接受状态——于是 JSON 被截断。理论上可以在生成时统计「还差多少 token 才能收尾」，但工业实现里并不总是做这件事。知道这个边界，面试和工程排查时都用得上。

6.5 本章小结

工具调用的合法性分语法、形状、类型三层，都能由 JSON schema 描述；JSON 需要上下文无关文法，识别它用下推自动机；xGrammar 通过对 logits 做掩码实现「生成即合法」，其唯一边界是 token 预算耗尽导致的截断。

7 工具接口的工程化：REST API 与 MCP

7.1 三种把工具接进 agent 的方式

1. **程序化调用**：给 agent 一个 Python 环境和一个库（如 `weatherlib.py`），让它写代码调用函数。简单直接，但不能远程调用，也不兼容标准工具调用通道。
2. **REST API**：用 FastAPI 之类的框架把函数暴露成 HTTP 接口，配一个 API key，写一个装饰器就把函数变成端点。好处是：可以用 key 限制访问；而且整个 API 会自动生成 JSON schema，可以直接交给模型当工具定义。coding agent 若能用 Bash，也可以直接用 `curl` 调 REST 接口。
3. **MCP (Model Context Protocol)**：由 Anthropic 提出的「给模型提供上下文/工具」的标准协议，架构是 host（AI 应用）连接多个 client，每个 client 再连一个 MCP server（本地 stdio 或远程 socket）。

7.2 MCP 真正的价值：多一层凭证隔离

主讲人坦言自己最初不理解 MCP 的必要性：既然 REST 已经能调 API，为什么还要多一个「跑程序的协议」？答案在于**安全边界**。MCP 有两层 key：

- **MCP API key**：交给运行 agent 的进程，用来连 MCP server；
- **上游 API key**（如 GitHub token）：只交给 MCP server，不交给 agent。

这样即使 agent 把 MCP key 泄露了，泄露的东西也基本无害；真正的上游凭证始终不在 agent 手里。反例很好记：如果把 GitHub token 直接给 agent，它可能某天「灵机一动」把 token 推到公开仓库，账号与所有仓库就都沦陷了。除了凭证隔离，MCP 与 REST 还在发现方式（OpenAPI 文档 vs 专门的 list tools 接口）、调用传输（HTTP vs stdio/socket）、能力范围（resources/prompts/extensions）上不同；生态上已有官方 MCP registry 和大量聚合站点，FastMCP 则让「把 REST 快速变成 MCP server」变得很容易。

7.3 本章小结

工具接入有三种方式：程序化调用最直接、REST 最通用、MCP 最利于权限隔离。MCP 的核心价值不是「多一层协议」，而是把上游凭证留在 server 侧、只把无害的代理 key 交给 agent。

8 工具使用的评测：把「会调工具」变成可测量的指标

工具调用是 agent 能力的前置条件，因此需要单独评测。最著名的是 **Berkeley Function Calling Leaderboard (BFCL)**，从 v1 到 v4 逐步覆盖：单轮工具调用 → 多轮工具调用 → agentic 工具调用（多步任务）→ 鲁棒性（参数幻觉、格式敏感性等）。

评测时要分维度看「整条工具使用链路」：

1. **选对工具**：面对任务，选择的工具是否真的能完成它；
2. **参数正确**：参数值是否正确（包括单位、格式）；
3. **顺序正确**：多步调用里次序是否合理；

4. **端到端准确率**：任务最终是否完成；

5. **效率**：完成需要多少 token、多少时延、多少成本；

6. **可靠性**：超时是否重试、工具失败能否换一个可用工具；

7. **安全性**：面对对抗性输入，是否会被诱导做出不当调用。

课堂上还展示了一个非常实用的对比：同一个模型（字幕称 GLM 5.3，具体型号以原视频为准），换推理供应商后工具调用错误率可以从约 15% 落到 0.01–0.05%。原因包括：不同供应商的解码/推理实现不同；量化等级不同（主讲人的经验是 FP8 明显好于 FP4，FP4 省成本但更像「另一个模型」）；投机解码是否无损；是否实现了约束解码（vLLM、SGLang 支持语法约束，有些供应商没用）；以及供应商本身的可靠性（服务挂掉时调用自然失败）。这条对工程选型意义很直接：**选模型不仅是选权重，也是选推理栈**。

8.1 本章小结

工具使用可以拆成七个可测维度（选择、参数、顺序、端到端、效率、可靠性、安全）；BFCL 是标准基准；同一模型在不同推理供应商上的工具调用可靠性差异巨大，量化、投机解码和约束解码实现是主因。

9 总结与延伸

本讲的收尾把工具使用概括成一个分层系统：工具为 agent 增加检索、执行、生成、应用等能力，是 agent 之所以为 agent 的关键；但这一切不是免费的——你需要把 schema 放进请求、让模型生成调用、解析并校验、分派执行、按 ID 配对结果，还可以用语法约束解码保证格式合法；而语法合法并不等于语义正确（一段能被解析的 Python 仍然可能算错）。接口层面有 REST 与 MCP 两条路，编排层面则要处理依赖、并发与评测。

把本讲压缩成五句话

1. 工具让语言模型从「生成文本」变成「对外部世界采取行动」；收益分扩展能力与便利化两类。

2. 工具调用有三种范式：函数清单、代码即元工具（CodeAct）、并行调用；代码表达力最强但必须沙箱化。

3. 工具调用是 token 序列，格式由各模型自己的协议决定；微调要对齐原生模板。

4. 执行侧是「解析、校验、分派、执行、回填」的流水线，call ID 必须与结果配对。

5. 受限解码（如 xGrammar）能让非法 JSON 无法生成，但只是语法约束；接口用 REST 或 MCP，评测看 BFCL 与多维度指标。

课程预告下一讲进入「长上下文建模」：既然工具会不断往上下文里塞观察，如何管理这些越来越长的上下文就是下一个核心问题。作业方面，第一次作业要求结合本讲写一段反思，并在 Canvas 上提交。

延伸思考：(1) 从训练视角看，工具层其实就是 RL 的「环境」——工具 schema、错误信息、沙箱边界都会影响 rollout 的质量与可验证性；(2) 从评测视角看，BFCL 这类基准测的是「工具使

用」而不是「完成任务」，两者之间还有很大 gap，agent 的端到端评测仍然更难；(3) 从工程视角看，MCP 的凭证隔离思路值得推广到所有「让模型接触真实系统」的场景——把危险能力留在服务端，只给模型最小权限。